DBMS LAB 9

Siddharth Karmokar

- 123CS0061

- ## Part A – JOINS (SQL Lab Exercise 1)

1. Display the names of customers who have placed orders along with the products they purchased using an INNER JOIN.

```
create table customer(
    cusid number primary key,
    custname varchar2(100),
    city varchar2(100)
);

insert into customer values (101, 'ramesh', 'hyderabad');
insert into customer values (102, 'suresh', 'chennai');
insert into customer values (103, 'mahesh', 'mumbai');
insert into customer values (104, 'naresh', 'delhi');
```

cript Output × | Query Result × | Query Result 1 ×

SQL | All Rows Fetched: 4 in 0.356 seconds

	CUSID	CUSTNAME	CITY
1	101	ramesh	hyderabad
2	102	suresh	chennai
3	103	mahesh	mumbai
4	104	naresh	delhi

```
insert into orders values(1, 101, 'laptop', 65000);
insert into orders values(2, 102, 'mouse', 500);
insert into orders values(3, 103, 'printer', 7000);
insert into orders values(4, 105, 'keyboard', 900);

select * from orders;
select * from customer;
```

cript Output × | Query Result × | Query Result 1 × | Query Result 2 × | Query Result 3 ×

SQL | All Rows Fetched: 4 in 0.005 seconds

	ORDERID	CUSID	PRODUCT	AMOUNT
1	1	101	laptop	65000
2	2	102	mouse	500
3	3	103	printer	7000
4	4	105	keyboard	900

```
insert into sales values(1, 'arjun', null);
insert into sales values(2, 'varun', 1);
insert into sales values(3, 'kiran', 1);
insert into sales values(4, 'manoj', 2);

select * from sales;
select * from orders;
select * from customer;
```

ipt Output × | Query Result × | Query Result 1 × | Query Result 2 × | Query Result 3 × | Query Result 4 ×

SQL | All Rows Fetched: 4 in 0.007 seconds

	SALESID	SALESNAME	MANAGERID
1	1	arjun	(null)
2	2	varun	1
3	3	kiran	1
4	4	manoj	2

```
insert into sales values(1, 'arjun', null);
insert into sales values(2, 'varun', 1);
insert into sales values(3, 'kiran', 1);
insert into sales values(4, 'manoj', 2);

select * from sales;
select * from orders;
select * from customer;
```

ipt Output × | Query Result × | Query Result 1 × | Query Result 2 × | Query Result 3 × | Query Result 4 ×

SQL | All Rows Fetched: 4 in 0.007 seconds

	SALESID	SALESNAME	MANAGERID
1	1	arjun	(null)
2	2	varun	1
3	3	kiran	1
4	4	manoj	2

```
select c.custname, o.product
from customer c
inner join orders o
on c.cusid = o.cusid;
```

Script Output × | Query Result × | Query Result 1 × | Query Result 2 × | Query Result 3 × | Query Result 4 × | Query Result 5 ×

SQL | All Rows Fetched: 3 in 0.008 seconds

	CUSTNAME	PRODUCT
1	ramesh	laptop
2	suresh	mouse
3	mahesh	printer

2. Display all customers and their orders (if any) using a LEFT OUTER JOIN. Include customers who have not placed any order.

```
select c.custname, o.product
from customer c
left outer join orders o
on c.cusid = o.cusid;
```

Query Result 5 — All Rows Fetched: 4 in 0.004 seconds

	CUSTNAME	PRODUCT
1	ramesh	laptop
2	suresh	mouse
3	mahesh	printer
4	naresh	(null)

3. Display all orders and the names of the customers who placed them using a RIGHT OUTER JOIN. Include orders placed by customers not found in the CUSTOMER table.

```
select c.custname, o.product
from customer c
right outer join orders o
on c.cusid = o.cusid;
```

Query Result 5 — All Rows Fetched: 4 in 0.003 seconds

	CUSTNAME	PRODUCT
1	ramesh	laptop
2	suresh	mouse
3	mahesh	printer
4	(null)	keyboard

4. Display all customers and all orders using a FULL OUTER JOIN. Include customers without orders and orders without matching customers.

```
select c.custname, o.product
from customer c
full outer join orders o
on c.cusid = o.cusid;
```

Query Result 5 — All Rows Fetched: 5 in 0.003 seconds

	CUSTNAME	PRODUCT
1	ramesh	laptop
2	suresh	mouse
3	mahesh	printer
4	(null)	keyboard
5	naresh	(null)

5. Display every possible combination of customers and products using a CROSS JOIN.

```
select c.custname, o.product
from customer c
cross join orders o;
```

Query Result 5 — All Rows Fetched: 16 in 0.005 seconds

	CUSTNAME	PRODUCT
1	ramesh	laptop
2	suresh	laptop
3	mahesh	laptop
4	naresh	laptop
5	ramesh	mouse
6	suresh	mouse
7	mahesh	mouse
8	naresh	mouse
9	ramesh	printer
10	suresh	printer
11	mahesh	printer
12	naresh	printer
13	ramesh	keyboard
14	suresh	keyboard
15	mahesh	keyboard
16	naresh	keyboard

6. Using the SALES table, display each employee along with their manager's name using a SELF JOIN.

```
select A.salesid, A.salesname, B.salesname as managername
from sales A, sales B
where A.salesid = B.managerid;
```

Query Result 5 — All Rows Fetched: 3 in 0.003 seconds

	SALESID	SALESNAME	MANAGERNAME
1	1	arjun	varun
2	1	arjun	kiran
3	2	varun	manoj

- Part B – VIEWS (SQL Lab Exercise 1)

1. Create a view that displays customer names and cities located in ‘Hyderabad’.

```
create view cusv as
select c.custname, c.city
from customer c
where c.city = 'hyderabad';

select * from cusv;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

All Rows Fetched: 1 in 0.001 seconds

	CUSTNAME	CITY
1	ramesh	hyderabad

2. Display all data from the view created above.

```
select * from customer;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

All Rows Fetched: 4 in 0.001 seconds

	CUSID	CUSTNAME	CITY
1	101	ramesh	mumbai
2	102	suresh	chennai
3	103	mahesh	mumbai
4	104	naresh	delhi

3. Update the city of a customer (choose any one) using the view.

```
select * from cusv;

update cusv
set city = 'mumbai';
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

All Rows Fetched: 0 in 0 seconds

CUSTNAME	CITY

```
update customer
set city = 'hyderabad'
where cusid = 101 or cusid = 104;

select * from cusv;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

All Rows Fetched: 2 in 0.001 seconds

	CUSTNAME	CITY
1	ramesh	hyderabad
2	naresh	hyderabad

4. Delete a customer record from the view whose city matches a specific value (choose any).

```
delete from cusv
where custname = 'naresh';
```

5. Drop the view created earlier.

```
View CUSV dropped.
```

6. Create a view that shows customer name, product, and order amount by joining the CUSTOMER and ORDERS tables.

7. Display all data from the join view created above.

```
create view cusvv as
select c.custname, o.product, o.amount
from customer c
inner join orders o
on c.cusid = o.cusid;

select * from cusvv;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6

All Rows Fetched: 3 in 0.002 seconds

	CUSTNAME	PRODUCT	AMOUNT
1	ramesh	laptop	65000
2	suresh	mouse	500
3	mahesh	printer	7000

- Part A – JOINS (SQL Lab Exercise 2)

1. Display the names of members and the titles of the books they have borrowed using an INNER JOIN.

```
select m.membername, b.title
from members m
inner join issue_details f
on m.memberid = f.memberid
inner join books b
on f.bookid = b.bookid;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 3 in 0.008 seconds

	MEMBERNAME	TITLE
1	arjun	database systems
2	arjun	let us c
3	sneha	python basics

2. Display all members and the books they have borrowed (if any) using a LEFT OUTER JOIN. Include members who have not borrowed any books.

```
select m.membername, b.title
from members m
left outer join issue_details i
on m.memberid = i.memberid
left outer join books b
on i.bookid = b.bookid;

select * from members;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 5 in 0.004 seconds

	MEMBERNAME	TITLE
1	arjun	database systems
2	arjun	let us c
3	sneha	python basics
4	rohit	(null)
5	neha	(null)

3. Display all book titles and the members who have borrowed them using a RIGHT OUTER JOIN. Include books that have not been borrowed.

```
select b.title, m.membername
from books b
left outer join issue_details i
on b.bookid = i.bookid
left outer join members m
on m.memberid = i.memberid;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 4 in 0.004 seconds

	TITLE	MEMBERNAME
1	database systems	arjun
2	let us c	arjun
3	python basics	sneha
4	data mining	(null)

4. Display all members and all books using a FULL OUTER JOIN. Include members without borrowed books and books without any borrower.

```
select m.membername, b.title
from members m
full outer join issue_details i
on m.memberid = i.memberid
full outer join books b
on i.bookid = b.bookid;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 6 in 0.003 seconds

	MEMBERNAME	TITLE
1	arjun	database systems
2	sneha	python basics
3	arjun	let us c
4	(null)	data mining
5	rohit	(null)
6	neha	(null)

5. Display every possible combination of members and books using a CROSS JOIN.

```
select m.membername, b.title
from members m
cross join issue_details i
cross join books b;

select * from members;
```

All Rows Fetched: 64 in 0.007 seconds

	MEMBERNAME	TITLE
1	arjun	database systems
2	arjun	database systems
3	arjun	database systems
4	arjun	database systems
5	arjun	let us c
6	arjun	let us c
7	arjun	let us c
8	arjun	let us c
9	arjun	python basics
10	arjun	python basics
11	arjun	python basics
12	arjun	python basics
13	arjun	data mining
14	arjun	data mining
15	arjun	data mining
16	arjun	data mining
17	sneha	database systems
18	sneha	database systems
19	sneha	database systems
20	sneha	database systems
21	sneha	let us c
22	sneha	let us c
23	sneha	let us c
24	sneha	let us c
25	sneha	python basics
26	sneha	python basics
27	sneha	python basics
28	sneha	python basics

6. Using the STAFF table, display each staff member along with their supervisor's name using a SELF JOIN.

```
select a.staffid, a.staffname, b.staffname as supervisorname
from staff a, staff b
where a.supervisorid = b.staffid;
```

All Rows Fetched: 3 in 0.004 seconds

	STAFFID	STAFFNAME	SUPERVISORNAME
1	s2	ramesh	meena
2	s3	kavya	meena
3	s4	mohan	ramesh

- # Part B – VIEWS (SQL Lab Exercise 2)

1. Create a view that displays book titles and their authors where the price is greater than 500.

```
create view baa as
select b.title, b.author, b.price
from books b
where b.price > 500;
```

All Rows Fetched: 3 in 0.002 seconds

	TITLE	AUTHOR	PRICE
1	database systems	navathe	550
2	python basics	reema thareja	600
3	data mining	han and kamber	700

```
update baa
set price = price + 600
where author = 'han and kamber';

select * from baa;
```

All Rows Fetched: 3 in 0.001 seconds

	TITLE	AUTHOR	PRICE
1	database systems	navathe	550
2	python basics	reema thareja	600
3	data mining	han and kamber	1300

2. Display all the data from the view created above.

```
delete from baa
where price > 600;

select * from baa;
```

All Rows Fetched: 2 in 0.001 seconds

	TITLE	AUTHOR	PRICE
1	database systems	navathe	550
2	python basics	reema thareja	600

3. Update the price of a specific book (choose any title) using the view.
4. Delete a record from the view for a book priced above 600.

5. Drop the view created earlier.

```
drop view baaa;
```

Script Output × | Query Result ×

Task completed in 0.354 seconds

6. Create a view that displays MemberName, BookTitle, and IssueDate by joining MEMBERS, BOOKS, and ISSUE_DETAILS tables.
7. Display all data from the join view created above.

```
create view baaa as
select m.membername, b.title, i.issuedate
from members m
inner join issue_details i
on m.memberid = i.memberid
inner join books b
on b.bookid = i.bookid;

select * from baaa;
```

Script Output × | Query Result × | Query Result 1 ×

SQL | All Rows Fetched: 3 in 0.006 seconds

	MEMBERNAME	TITLE	ISSUEDATE
1	arjun	database systems	01-09-24
2	arjun	let us c	04-09-24
3	sneha	python basics	03-09-24